Topic 2: Divide-and-Conquer Algorithms

#eda #apunts #english #PARCIAL1

1. The Divide-and-Conquer Paradigm

Divide and Conquer solves a computational problem through a three-step recursive framework:

1. **Divide**: Partition the problem of size n into $a \geq 1$ subproblems of smaller size n/b (where $b > 1$).
2. **Conquer**: Recursively solve each subproblem.
3. **Combine**: Recombine the subproblem solutions into a complete solution for the original problem.

Standard Cost Recurrence

When dividing and recombining solutions takes $\Theta(n^k)$ time (for $k \geq 0$), the overall cost function $T(n)$ satisfies the recurrence:

$$T(n) = a \cdot T(n/b) + \Theta(n^k)$$

This recurrence is evaluated using **Master Theorem II**:

- **Case 1**: $\log_b(a) < k \implies T(n) \in \Theta(n^k)$.
- **Case 2**: $\log_b(a) = k \implies T(n) \in \Theta(n^k \log n)$.
- **Case 3**: $\log_b(a) > k \implies T(n) \in \Theta(n^{\log_b a})$.

---

2. Searching Algorithms

Binary Search

- **Mechanism**: Given a sorted vector, compare target element v with the middle element. If v is smaller, recursively search the left subvector; if larger, search the right subvector.
- **Recurrence**: $T(n) = T(n/2) + \Theta(1)$.
- **Complexity**: $T(n) \in \Theta(\log n)$ via Master Theorem II ($\alpha = \log_2 1 = 0 = k$).
- **Explanation:**
  As we only have 1 instruction per recurrence step $\Theta(n^k) = \Theta(1) \implies k = 0$, also, as we only create a new subproblem $a = 1$, and that subproblem is the half of the vector $b = 2 \implies T(n) = 1 \times T(n/2) + \Theta(1)$.
  Then by Master Theorem II we find that $\log_2 1 = 0 = k = 0 \implies T(n) \in \Theta(n^k \log n) = \Theta(1 \log n) = \Theta(\log n)$

---

3. Sorting Algorithms

Merge Sort

- **Invented by**: John von Neumann (1945).
- **Mechanism**:
  1. Split the vector into two equal halves.
  2. Recursively sort both halves.
  3. Combine the two sorted halves using a linear-time `merge` routine in $\Theta(n)$ time.
- **Properties**:
  - **Stable**: Preserves the original relative order of equal elements.
  - **Consistent**: Operates in $\Theta(n \log n)$ regardless of initial sortedness level.
- **Recurrence**:

$$T(1) = \Theta(1)$$

$$T(n) = 2T(n/2) + \Theta(n) \quad \text{for } n > 1 \implies T(n) \in \Theta(n \log n) \text{ (allcases)}$$

- **Variants**:
  - *Small-Vector Cutoff*: Switch to **Insertion Sort** when subvector size falls below a critical threshold (e.g., $n < 50$) to reduce recursion stack overhead.
  - *Iterative / Bottom-Up*: Merge adjacent single elements into pairs, then blocks of size 2, 4, 8, etc., using a queue or double loops.

QuickSort

- **Invented by**: C. A. R. Hoare (1960).
- **Mechanism**:
  1. Select a **pivot** element x.
  2. **Partition** vector T in-place into subvectors T_1 (elements $\leq x$) and T_2 (elements $\geq x$) in $\Theta(n)$ time.
  3. Recursively sort T_1 and T_2.
- **Pivot Selection Strategies**:
  1. *First Element*: Simple, but degrades to $\Theta(n^2)$ on already sorted or reverse-sorted inputs.
  2. *Random Element*: Safe strategy preventing adversarial worst-case inputs.
  3. *Median-of-Three*: Takes the median of the first, middle, and last elements.
- **Complexity Analysis**:
  - **Worst Case**: Pivot is always the minimum or maximum element $\implies T(n) = T(n-1) + \Theta(n) \in \Theta(n^2)$.
  - **Best Case**: Pivot splits vector into equal halves $\implies T(n) = 2T(n/2) + \Theta(n) \in \Theta(n \log n)$.
  - **Average Case**: Assuming uniform random pivot choice $\implies T(n) \in \Theta(n \log n)$.

```
int partition(vector<int>& T, int l, int r){
    int x = T[l];
    int i = l - 1;
    int j = r + 1;

    while(1){
        while (x < T[--j]);
        while (x > T[++i]);
        if (i >= j) return j;
        swap(T[i], T[j]);
    }
}
```

- **Goal**: Find the k-th smallest element in an unsorted list.
- **Mechanism**: Partition array around a pivot x into S_E ($\leq x$) and S_D ($> x$). Recursively call `selection` on only **one** subvector depending on whether $k \leq |S_E|$.
- **Complexity**: Worst-case $\Theta(n^2)$, Average-case $\Theta(n)$.

```
int quickSelect(const vector<int>& T, int k){
    if (T.size() == 1) return T[0];

    int pivot = T[0];
    vector<int> l;
    vector<int> r;
    for (int i = 1; i < T.size(); i++){
        if (T[i] <= pivot) l.push_back(T[i]);
        else r.push_back(T[i]);
    }
    if (k <=l.size()) return quickSelect(l, k);
    else if(k == l.size() + 1) return pivot;
    else return quickSelect(r, k - l.size() - 1);
}
```

Deterministic Linear Selection (Median-of-Medians)

- **Goal**: Guarantee worst-case $\Theta(n)$ time for finding the median or k-th smallest element.
- **Mechanism**:
  1. Divide vector into blocks of size q (typically $q = 5$).
  2. Find the median of each block in $\Theta(1)$ time per block ($\Theta(n)$ total).
  3. Recursively call selection to find the median of the n/q block medians (the *pseudo-median*).
  4. Use the pseudo-median as a pivot to partition the original vector.
  5. Recursively solve on the target subvector (bounded by size at most $\frac{3n}{4}$).

- **Recurrence**:

$$C(n) = C(n/q) + C(3n/4) + \Theta(n)$$

  Yields $C(n) \in \Theta(n)$ whenever $\frac{1}{q} + \frac{3}{4} < 1$ (e.g., $q = 5$).

---

5. Mathematical & Matrix Algorithms

Quick Exponentiation

- **Goal**: Compute x^n in logarithmic time.
- **Mechanism**:
  - If n is even: $x^n = (x^{n/2})^2$.
  - If n is odd: $x^n = x \cdot (x^{(n-1)/2})^2$.
- **Recurrence**: $T(n) = T(n/2) + \Theta(1) \implies T(n) \in \Theta(\log n)$.

```
int quickExponentiation(int x, int n){
    if (n == 0) return 1;
    int y = quickExponentiation(x, n / 2)
    if (n % 2 == 0) return  y * y; // (x^n/2)^2
    return y * y * x;              // (x^n-1/2)^2
}
```

Karatsuba's Integer Multiplication

- **Goal**: Multiply two n-bit integers faster than the standard school algorithm ($\Theta(n^2)$).
- **Gauss's Trick**: Reduces 4 subproducts of $n/2$-bit numbers to 3 subproducts:

$$a = x_{E\!E}, \quad b = x_{D\!D}, \quad c = (x_E + x_D)(\!E + \!D)$$

$$x \cdot \!= 2^n a + 2^{n/2}(c - a - b) + b$$

- **Recurrence**: $T(n) = 3T(n/2) + \Theta(n)$.
- **Complexity**: $T(n) \in \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.

Strassen's Matrix Multiplication

- **Goal**: Multiply two $n \times n$ matrices faster than standard $\Theta(n^3)$.
- **Mechanism**: Decompose $n \times n$ matrices into four $n/2 \times n/2$ subblock matrices. Compute 7 matrix subproducts ($P_1 \dots P_7$) instead of 8.
- **Recurrence**: $T(n) = 7T(n/2) + \Theta(n^2)$.
- **Complexity**: $T(n) \in \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$.

---

6. Classic Puzzles

Towers of Hanoi

- **Goal**: Move n disks from rod 1 to rod 2 using rod 3 as auxiliary.
- **Mechanism**:
  1. Move $n-1$ disks from rod 1 to rod 3.
  2. Move the largest disk from rod 1 to rod 2.
  3. Move $n-1$ disks from rod 3 to rod 2.
- **Recurrence**: $T(0) = 0, \quad T(n) = 2T(n-1) + 1$.
- **Exact Closed-Form Solution**: $T(n) = 2^n - 1 \implies \Theta(2^n)$.

```
void hanoi(int n, int a, int b, int c) {
    if (n > 0) {
        hanoi(n - 1, a, c, b); // Line 1: Move n-1 disks out of the way
        cout << a << " -> " << b << endl; // Line 2: Move the big disk to
target
        hanoi(n - 1, c, b, a); // Line 3: Move n-1 disks onto the big
disk
    }
}
```